

part B. The Type System

What's in a Type?

A “type” is a description of what a JavaScript value *shape* might be. By “shape” I mean which properties and methods exist on a value, and what the built-in `typeof` operator would describe it as.

For example, when you create a variable with the initial value `"Aretha"`:

```
let singer = "Aretha";
```

TypeScript can infer, or figure out, that the `singer` variable is of *type* string.

The most basic types in TypeScript correspond to the seven basic kinds of primitives in JavaScript:

- `null`
- `undefined`
- `boolean` // `true` or `false`
- `string` // `""`, `"Hi!"`, `"abc123"`, ...
- `number` // `0`, `2.1`, `-4`, ...
- `bigint` // `0n`, `2n`, `-4n`, ...
- `symbol` // `Symbol()`, `Symbol("hi")`, ...

For each of these values, TypeScript understands the type of the value to be one of the seven basic primitives:

- `null; // null`
- `undefined; // undefined`
- `true; // boolean`
- `"Louise"; // string`
- `1337; // number`
- `1337n; // bigint`
- `Symbol("Franklin"); // symbol`

If you ever forget the name of a primitive, you can type a `let` variable with a primitive value into the [TypeScript Playground](#) or an IDE and hover your mouse over the

variable's name. The resultant popover will include the name of the primitive, such as this screenshot showing hovering over a string variable .

```
2
3
4   let singer: string
5   let singer = "Ella Fitzgerald";
6
```

Figure 2-1. TypeScript showing a string variable's type in its hover information

TypeScript is also smart enough to be able to infer the type of a variable whose starting value is computed. In this example, TypeScript knows that the ternary expression always results in a string, so the `bestSong` variable is a `string`:

```
// Inferred type: string
let bestSong = Math.random() > 0.5
  ? "Chain of Fools"
  : "Respect";
```

Back in the [TypeScript Playground](#) or your IDE, try hovering your cursor on that `bestSong` variable. You should see some info box or message telling you that TypeScript has inferred the `bestSong` variable to be type `string`

```
let bestSong: string
let bestSong = Math.random() > 0.5
  ? "Chain of Fools"
  : "Respect";
```

Figure 2-2. TypeScript reporting a `Let` variable as being its `string` literal type from its ternary expression

Type Systems

A *type system* is the set of rules for how a programming language understands what types the constructs in a program may have.

At its core, TypeScript's type system works by:

- Reading in your code and understanding all the types and values in existence
- For each value, seeing what type its initial declaration indicates it may contain
- For each value, seeing all the ways it's used later on in code

- Complaining to the user if a value’s usage doesn’t match with its type

Let’s walk through this type inference process in detail.

Take the following snippet, in which TypeScript is emitting a type error about a member property being erroneously called as a function:

```
let firstName = "Whitney";
firstName.length();
//      ~~~~~~
// This expression is not callable.
//   Type 'Number' has no call signatures.
```

TypeScript came to that complaint by, in order:

1. Reading in the code and understanding there to be a variable named `firstName`
2. Concluding that `firstName` is of type `string` because its initial value is a `string`, `"Whitney"`
3. Seeing that the code is trying to access a `.length` member of `firstName` and call it like a function
4. Complaining that the `.length` member of a string is a number, not a function (*it can’t be called like a function*)

Understanding TypeScript’s type system is an important skill for understanding TypeScript code. Code snippets in this part and throughout the rest of this part will display more and more complex types that TypeScript will be able to infer from code.

Kinds of Errors

While writing TypeScript, the two kinds of “errors” you’ll come across most frequently are:

Syntax

Blocking TypeScript from being converted to JavaScript

Type

Something mismatched has been detected by the type checker

The differences between the two are important.

Syntax errors

Syntax errors are when TypeScript detects incorrect syntax that it cannot understand as code. These block TypeScript from being able to properly generate output JavaScript from your file. Depending on the tooling and settings you're using to convert your TypeScript code to JavaScript, you might still get some kind of JavaScript output (in default `tsc` settings, you will). But if you do, it likely won't look like what you expect.

This input TypeScript has a syntax error for an unexpected `let`:

```
let let wat;  
//      ~~~  
// Error: ' ,' expected.
```

Its compiled JavaScript output, depending on the TypeScript compiler version, may look something like:

```
let let, wat;
```

Type errors

Type errors occur when your syntax is valid but the TypeScript type checker has detected an error with the program's types. These do not block TypeScript syntax from being converted to JavaScript. They do, however, often indicate something will crash or behave unexpectedly if your code is allowed to run.

You saw this in with the `console.blub` example, where the code was syntactically valid but TypeScript could detect it would likely crash when run:

```
console.blub("Nothing is worth more than laughter.");  
//      ~~~~  
// Error: Property 'blub' does not exist on type 'Console'.
```

Even though TypeScript may output JavaScript code despite the presence of type errors, type errors are generally a sign

that the output JavaScript likely won't run the way you wanted. It's best to read them and consider fixing any reported issues before running JavaScript.

Assignability

TypeScript reads variables' initial values to determine what type those variables are allowed to be. If it later sees an assignment of a new value to that variable, it will check if that new value's type is the same as the variable's.

TypeScript is fine with later assigning a different value of the same type to a variable. If a variable is, say, initially a `string` value, later assigning it another `string` would be fine:

```
let firstName = "Carole";
firstName = "Joan";
```

If TypeScript sees an assignment of a different type, it will give us a type error. We couldn't, say, initially declare a variable with a `string` value and then later on put in a `boolean`:

```
let lastName = "King";
lastName = true;
// Error: Type 'boolean' is not assignable to type 'string'.
```

TypeScript's checking of whether a value is allowed to be provided to a function call or variable is called *assignability*: whether that value is *assignable* to the expected type it's passed to. This will be an important term in later parts as we compare more complex objects.

Understanding Assignability Errors

Errors in the format “Type...is not assignable to type...” will be some of the most common types of errors you'll see when writing TypeScript code.

The first type mentioned in that error message is the value the code is attempting to assign to a recipient. The second type mentioned is the recipient being assigned the first type. For example, when we wrote `lastName = true` in the previ-

ous snippet, we were trying to *assign* the value of `true` — type `boolean` — to the recipient variable `lastName` — type `string`.

You'll see more and more complex assignability issues as you progress through this part. Remember to read them carefully to understand reported differences between actual and expected types. Doing so will make it much easier to work with TypeScript when it's giving you grief over type errors.

Type Annotations

Sometimes a variable doesn't have an initial value for TypeScript to read. TypeScript won't attempt to figure out the initial type of the variable from later uses. It'll consider the variable by default to be implicitly the `any` type: indicating that it could be anything in the world.

Variables that can't have their initial type inferred go through what's called an *evolving any*: rather than enforce any particular type, TypeScript will evolve its understanding of the variable's type each time a new value is assigned.

Here, assigning the evolving `any` variable `rocker` is first assigned a string, which means it has string methods such as `toUpperCase`, but then is evolved into a `number`:

```
let rocker; // Type: any

rocker = "Joan Jett"; // Type: string
rocker.toUpperCase(); // Ok

rocker = 19.58; // Type: number
rocker.toPrecision(1); // Ok

rocker.toUpperCase();
//      ~~~~~
// Error: 'toUpperCase' does not exist on type 'number'.
```

TypeScript was able to catch that we were calling the `toUpperCase()` method on a variable evolved to type `number`. However, it wasn't able to tell us earlier whether it was intentional that we were evolving the variable from `string` to `number` in the first place.

Allowing variables to be evolving `any` typed—and using the `any` type in general—partially defeats the purpose of TypeScript’s type checking! TypeScript works best when it knows what types your values are meant to be. Much of TypeScript’s type checking can’t be applied to `any` typed values because they don’t have known types to be checked. will cover how to configure TypeScript’s implicit `any` complaints.

TypeScript provides a syntax for declaring the type of a variable without having to assign it an initial value, called a *type annotation*. A type annotation is placed after the name of a variable and includes a colon followed by the name of a type.

This type annotation indicates the `rocker` variable is meant to be type `string`:

```
let rocker: string;
rocker = "Joan Jett";
```

These type annotations exist only for TypeScript—they don’t affect the runtime code and are not valid JavaScript syntax. If you run `tsc` to compile TypeScript source code to JavaScript, they’ll be erased. For example, the previous example would be compiled to roughly the following JavaScript:

```
// output .js file
let rocker;
rocker = "Joan Jett";
```

Assigning a value whose type is not assignable to the variable’s annotated type will cause a type error.

This snippet assigns a number to a `rocker` variable previously declared as type `string`, causing a type error:

```
let rocker: string;
rocker = 19.58;
// Error: Type 'number' is not assignable to type 'string'.
```

You’ll see through the next few parts how type annotations allow you to augment TypeScript’s insights into your code,

allowing it to give you better features during development. TypeScript contains an assortment of new pieces of syntax, such as these type annotations that exist only in the type system.

Unnecessary Type Annotations

Type annotations allow us to provide information to TypeScript that it wouldn't have been able to glean on its own. You could also use them on variables that have immediately inferable types, but you wouldn't be telling TypeScript anything it doesn't already know.

The following `: string` type annotation is redundant because TypeScript could already infer that `firstName` be of type `string`:

```
let firstName: string = "Tina";  
// ~~~~~ Does not change the type system...
```

If you do add a type annotation to a variable with an initial value, TypeScript will check that it matches the type of the variable's value.

The following `firstName` is declared to be of type `string`, but its initializer is the `number` `42`, which TypeScript sees as an incompatibility:

```
let firstName: string = 42;  
// ~~~~~  
// Error: Type 'number' is not assignable to type 'string'.
```

Many developers—myself included—generally prefer not to add type annotations on variables where the type annotations wouldn't change anything. Having to manually write out type annotations can be cumbersome—especially when they change, and for the complex types I'll show you later in this material.

It can sometimes be useful to include explicit type annotations on variables to clearly document the code and/or to make TypeScript protected against accidental changes to the variable's type. We'll see in later parts how explicit type an-

notations can sometimes explicitly tell TypeScript information it wouldn't have inferred normally.

Type Shapes

TypeScript does more than check that the values assigned to variables match their original types. TypeScript also knows what member properties should exist on objects. If you attempt to access a property of a variable, TypeScript will make sure that property is known to exist on that variable's type.

Suppose we declare a `rapper` variable of type `string`. Later on, when we use that `rapper` variable, operations that TypeScript knows work on strings are allowed:

```
let rapper = "Queen Latifah";
rapper.length; // ok
```

Operations that TypeScript doesn't know to work on strings will not be allowed:

```
rapper.push('!');
//      ~~~~
// Property 'push' does not exist on type 'string'.
```

Types can also be more complex shapes, most notably objects. In the following snippet, TypeScript knows the `cher` object doesn't have a `middleName` key and complains:

```
let cher = {
  firstName: "Cherilyn",
  lastName: "Sarkisian",
};

cher.middleName;
//      ~~~~~~
// Property 'middleName' does not exist on type
//   '{ firstName: string; lastName: string; }'.
```

TypeScript's understanding of object shapes allows it to report issues with the usage of objects, not just assignability. will describe more of TypeScript's powerful features around objects and object types.

Modules

The JavaScript programming language did not include a specification for how files can share code between each other until relatively recently in its history. ECMAScript 2015 added “ECMAScript modules,” or ESM, to standardize `import` and `export` syntax between files.

For reference, this module file imports a `value` from a sibling `./values` file and exports a `doubled` variable:

```
import { value } from "./values";

export const doubled = value * 2;
```

To match with the ECMAScript specification, in this part I’ll use the following nomenclature:

Module

A file with a top-level `export` or `import`

Script

Any file that is not a module

TypeScript is able to work with those modern module files as well as older files. Anything declared in a module file will be available only in that file unless an explicit `export` statement in that file exports it. A variable declared in one module with the same name as a variable declared in another file won’t be considered a naming conflict (unless one file imports the other file’s variable).

The following `a.ts` and `b.ts` files are both modules that export a similarly named `shared` variable without issue. `c.ts` causes a type error because it has a naming conflict between an imported `shared` and its own value:

```
// a.ts
export const shared = "Cher";
```

```
// b.ts
export const shared = "Cher";
```

```
// c.ts
import { shared } from "./a";
//      ~~~~~~
// Error: Import declaration conflicts with local declaration of 'shared'.

export const shared = "Cher";
//      ~~~~~~
// Error: Individual declarations in merged declaration
// 'shared' must be all exported or all local.
```

If a file is a script, though, TypeScript will consider it to be globally scoped, meaning all scripts have access to its contents. That means variables declared in a script file cannot have the same name as variables declared in other script files.

The following `a.ts` and `b.ts` files are considered scripts because they do not have module-style `export` or `import` statements. That means their variables of the same name conflict with each other as if they were declared in the same file:

```
// a.ts
const shared = "Cher";
//      ~~~~~~
// Cannot redeclare block-scoped variable 'shared'.
```

```
// b.ts
const shared = "Cher";
//      ~~~~~~
// Cannot redeclare block-scoped variable 'shared'.
```

If you see these “Cannot redeclare...” errors in a TypeScript file, it may be because you have yet to add an `export` or `import` statement to the file. Per the ECMAScript specification, if you need a file to be a module without an `export` or `import` statement, you can add an `export {};` somewhere in the file to force it to be a module:

```
// a.ts and b.ts
const shared = "Cher"; // Ok

export {};
```

In this part, you saw how TypeScript’s type system works at its core:

- What a “type” is and the primitive types recognized by TypeScript
- What a “type system” is and how TypeScript’s type system understands code
- How type errors compare to syntax errors
- Inferred variable types and variable assignability
- Type annotations to explicitly declare variable types and avoid evolving `any` types
- Object member checking on type shapes
- ECMAScript module files’ declaration scoping compared to script files